

LECTURE 10

Systems in Product Development

Tools : System Archetypes

System Archetypes

System Archetypes are recurring patterns of behavior that appear again and again in different systems — technology, society, business, education, environment, and products.

They are like “mental templates” that help us quickly recognize why problems keep repeating.

Think of archetypes as system behavior blueprints.

Three Types of System Archetypes

Three of the most useful archetypes for engineering, tech, and product design.

- Fixes that Fail
- Tragedy of Commons
- Limits to Growth

Fixes That Fail

Core Idea : A quick solution temporarily solves a problem, but creates bigger problems later.

Short-term relief → Long-term damage

Structure Pattern

Problem → Quick Fix → Temporary Improvement → Unintended Consequence → Bigger Problem → More Quick Fixes

It becomes a vicious cycle

Fixes That Fail

Real-World Example 1 – App Notifications Overload

Problem: Low user engagement

Quick Fix: Send more push notifications

Short Term Result: Engagement increases.

Long Term Result: Users feel spammed, Notifications turned off, App uninstalled, Engagement drops worse than before.

System Insight:

The “fix” attacked the symptom, not the root cause (poor content or UX).

Fixes That Fail

Real-World Example 2 – Painkillers for Lifestyle Issues

Problem: Frequent headaches

Quick Fix: Regular painkiller use

Outcome:

- Temporary relief
- Dependency or health side effects
- Root cause (sleep, stress, posture) ignored.

Fixes That Fail

Product Design Lessons

If you only optimise surface metrics, the system fights back later.

Ask:

- What root cause am I ignoring?
- What delayed side effects may appear?

Tragedy of Commons

Core Idea : When individual users act in self-interest, they collectively destroy a shared resource.

Personal gain → Collective loss

Structure Pattern

Shared Resource → Individual Usage ↑ → Resource Depletion → System Collapse

Tragedy of Commons

Real-World Example 1 – Internet Bandwidth in Hostels

Context: Each student streams HD videos simultaneously

Individual View: “I just want good quality.”

Collective Result:

- Network slows for everyone
- Calls drop
- Frustration increases

Result: *No one intended harm, but the shared system degraded.*

Tragedy of Commons

Real-World Example 2 – Open-Source Maintainers Burnout

Context: Many developers use free libraries.

Issue:

- Few contribute back
- Maintainers overworked
- Projects abandoned

The shared “commons” is maintainer energy and time.

Tragedy of Commons

Product Design Lessons

Whenever you design platforms, APIs, shared tools, or ecosystems, ask:

- Who maintains the commons?
- Are incentives balanced?
- Is there a usage limit or governance?

Limits to Growth

Core Idea : A system grows rapidly at first, but hidden constraints eventually slow or stop growth.

Success contains the seeds of its own limits.

Structure Pattern

Growth Loop → Constraint Appears → Slowing Growth →
Plateau or Decline

Limits to Growth

Real-World Example 1 – Social Media Platform Growth

Early Stage:

- Rapid user growth
- Viral adoption

Limit Appears:

- Content moderation challenges
- Infrastructure cost
- Regulatory pressure

Growth slows or trust declines.

Limits to Growth

Engineering/Product Design Lessons

Scaling requires anticipating constraints like:

- Infrastructure capacity
- User support
- Governance
- Ethics
- Talent management
- Regulations

Growth is never infinite.